Week 6

Gymnasium, CartPole & FrozenLake
PPO and DQN in Practice

From Conway to LangGraph — Agent Systems for Physicists in the LLM Era

Mirko Degli Esposti · Dipartimento di Fisica · Università di Bologna

- Gymnasium API — the modern RL interface
- CartPole: inverted pendulum as control benchmark
- FrozenLake: sparse reward on a stochastic lattice
- DQN — Deep Q-Networks and experience replay
- PPO — Proximal Policy Optimisation
- Physics analogies: potential landscapes & Lyapunov stability

Session Road Map

4h lab

1. **Gymnasium API** — 30 min
   env, step(), reset() — the 5-tuple contract
2. **CartPole physics** — 15 min
   Inverted pendulum — state space & dynamics
3. **FrozenLake** — 15 min
   Sparse rewards — 4×4 lattice with holes
4. **DQN** — 25 min
   Deep Q-Network — replay buffer & target net
5. **PPO** — 25 min
   Proximal Policy Optimisation — clipped surrogate
6. **Training & analysis** — 60 min
   Reward landscapes, phase portrait, Lyapunov
7. **Algorithm comparison** — 20 min
   When to use PPO vs DQN — design criteria

The Gymnasium API — 5-Tuple Contract

gymnasium ≥ 0.26

Python

```python
import gymnasium as gym

env = gym.make('CartPole-v1')
obs, info = env.reset()

while True:
    action = env.action_space.sample()
    obs, reward, terminated,\
         truncated, info = env.step(action)

    if terminated or truncated:
        obs, info = env.reset()
        break
```

obs
observation
State vector — e.g. [x, ẋ, θ, θ̇] for CartPole

reward
scalar ∈ ℝ
+1 per step (CartPole), +1 at goal (FrozenLake)

terminated
bool
Episode ended by MDP rules (fall, hole, goal)

truncated
bool
Episode ended by time limit (500 steps)

info
dict
Auxiliary diagnostics — not used for training

⚠ Use `import gymnasium as gym` — not the deprecated `import gym` (OpenAI Gym). SB3 ≥ 2.0 requires Gymnasium (Farama Foundation).

CartPole — Inverted Pendulum on a Cart

4-Dimensional State Vector

Variable	Symbol	Physical meaning	Range
Cart position	x	Horizontal displacement	±2.4 m
Cart velocity	$\dot{x}$	Linear velocity	$\pm\infty$
Pole angle	θ	Angle from vertical (rad)	±0.21 rad
Angular velocity	$\dot{\theta}$	Angular velocity	$\pm\infty$

Termination: $|\theta| > 12°$ OR $|x| > 2.4$ m

Max reward per episode: 500 steps × (+1) = 500

PHYSICS ANALOGY

Linearised dynamics near upright equilibrium:

$$\ddot{\theta} = (g/l)\,\theta - (\cos\theta / ml^2)\,F$$

The optimal LQR policy is:

$$F = -k_1 x - k_2 \dot{x} - k_3 \theta - k_4 \dot{\theta}$$

PPO rediscovers this linear controller without ever seeing the dynamics model.

KEY INSIGHT

CartPole is an underactuated control problem — only 1 force (left/right push) to control 4 degrees of freedom. Dense reward (+1/step) means a gradient signal always exists.

FrozenLake — Sparse Reward on a Stochastic Lattice

S Start	F Frozen	F Frozen	F Frozen
F Frozen	H Hole	F Frozen	H Hole
F Frozen	F Frozen	F Frozen	H Hole
H Hole	F Frozen	F Frozen	G Goal

FrozenLake-v1 (4 × 4 grid)

STATE SPACE

16 discrete tiles ($|\mathcal{S}| = 16$)
Obs = tile index $\in \{0, \ldots, 15\}$

ACTION SPACE

4 discrete actions: ← ↓ → ↑
(is_slippery=True: only 1/3 chance of going that way)

REWARD

r = +1 at goal G
r = 0 everywhere else
→ Sparse reward landscape

PHYSICS ANALOGY

Stochastic FrozenLake = biased random walk on 2D lattice with absorbing boundaries
(Gambler's ruin / Dirichlet BVP)

DQN — Deep Q-Network

Mnih et al., Nature 2015

Replace the tabular Q-table from Week 5 with a neural network: $Q(s, a; \theta) \rightarrow$ Q-values for all actions in one forward pass.

Two Critical Innovations

① EXPERIENCE REPLAY

Store transitions (s, a, r, s') in a replay buffer $\mathcal{D}$.

At each step: sample a **random mini-batch** from $\mathcal{D}$ and minimise:

$$L(\theta) = \mathbb{E}[(r + \gamma \max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta))^2]$$

② TARGET NETWORK

Two networks: θ (online, trained every step) and θ^- (target, frozen for C steps).

Why? Without θ^-, the bootstrap target changes as fast as θ, creating a moving target — training diverges.

$$\theta^- \leftarrow \theta \quad \text{every C steps} \quad \text{(hard copy)}$$

Physics analogy — experience replay breaks temporal correlations between samples, just as disorder/annealing breaks ferromagnetic ordering: the network learns the average structure of the reward landscape, not just the most recent trajectory.

DQN for FrozenLake: Discrete actions → Q(s,a) outputs 4 values. Sparse reward → replay buffer sees rare goal transitions many times. Small state space → network effectively learns the tabular Q-table from Week 5.

DQN — Algorithm & Key Hyperparameters

DQN Pseudocode

```
Initialise  Q(s,a;θ)  and  Q(s,a;θ⁻) = Q(s,a;θ)
Initialise replay buffer  𝒟  (capacity N)

for episode = 1 … M:
  obs ← env.reset()
  for t = 1 … T:
    # ε-greedy exploration
    if rand() < ε:
      a ← random action
    else:
      a ← argmax_a Q(obs, a; θ)

    obs', r, done, _ ← env.step(a)
    Store (obs, a, r, obs', done) in 𝒟

    # SGD step on random mini-batch
    Sample {(sⱼ,aⱼ,rⱼ,s'ⱼ)} from 𝒟
    yⱼ = rⱼ + γ maxₐ Q(s'ⱼ,a;θ⁻) if not done
    Minimise  Σⱼ(yⱼ − Q(sⱼ,aⱼ;θ))²

    if t mod C == 0:  θ⁻ ← θ
```

Key Hyperparameters (SB3 defaults)

buffer_size	1 000 000	Max transitions stored
learning_rate	1e-4	Adam optimizer LR
batch_size	32	Mini-batch size
gamma (γ)	0.99	Discount factor
exploration_fraction	0.1	ε annealing window
exploration_final_eps	0.05	Final ε value
target_update_interval	10 000	Steps between $\theta^- \leftarrow \theta$
learning_starts	50 000	Steps before first SGD

```
from stable_baselines3 import DQN
model = DQN('MlpPolicy', env,
            verbose=0)
model.learn(total_timesteps=50_000)
```

PPO — Proximal Policy Optimisation

Schulman et al., 2017

Problem with vanilla policy gradient: a too-large update step can collapse the policy — and because the data is thrown away after each update, there is no cheap way to recover.

The PPO Solution — Trust Region via Clipping

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min\left(r_t(\theta)\,\hat{A}_t,\ \text{clip}\left(r_t(\theta), 1-\varepsilon, 1+\varepsilon \right) \hat{A}_t \right) \right]$$

$r_t(\theta) = \pi_\theta(a_t|s_t) / \pi_\theta_old(a_t|s_t)$ ← probability ratio · $\hat{A}_t$ ← advantage estimate · $\varepsilon \approx 0.2$ ← clip threshold

CLIPPING MECHANICS

If $\hat{A}_t > 0$ (good action): **r capped at 1+ε** — can't over-reward.

If $\hat{A}_t < 0$ (bad action): **r floored at 1−ε** — can't over-penalise.

Effect: policy update ≤ ε per step → stable training.

ON-POLICY ROLLOUTS

Collect n_steps rollouts with π_old, then:

- Compute advantages Â using GAE
- Run **n_epochs** SGD passes over the same batch
- Discard data and collect new rollouts

Data reuse within epoch is what makes PPO sample-efficient vs A2C.

Physics analogy — clipping is a trust region: update θ only within a ball of radius ε around θ_old. This mirrors the Renormalisation Group constraint: only integrate out modes within a shell, never the whole theory at once.

PPO — Hyperparameters & SB3 Code

CartPole-v1

Key Hyperparameters

n_steps	2048	Rollout buffer length before each update
batch_size	64	Mini-batch size per SGD step
n_epochs	10	Reuse factor — passes over each rollout
gamma (γ)	0.99	Discount factor (same as DQN)
gae_lambda (λ)	0.95	GAE trace decay — bias/variance tradeoff
clip_range (ε)	0.2	PPO clip threshold — trust region radius
ent_coef	0.0	Entropy bonus (0 = no exploration incentive)
learning_rate	3e-4	Adam LR (3× larger than DQN default)

SB3 — CartPole PPO training

```
import gymnasium as gym
from stable_baselines3 import PPO
from stable_baselines3.common
  .callbacks import EvalCallback

env = gym.make('CartPole-v1')

model = PPO(
    'MlpPolicy', env,
    n_steps=2048,
    batch_size=64,
    n_epochs=10,
    gamma=0.99,
    verbose=1
)

model.learn(
    total_timesteps=100_000
)
```

EXPECTED TRAINING OUTCOME (CartPole, 100k steps, CPU ~3-5 min)

Reaches mean reward > 450 at ~30-50k steps · Final performance: 500/500 (perfect) with deterministic eval · Convergence is smooth — characteristic of PPO's trust region

PPO vs DQN — Design Criteria

When to use which

Criterion	PPO	DQN
Action space	Continuous or discrete	Discrete only
Reward density	Dense (gradient always exists)	Sparse (replay handles rare $r > 0$)
Policy type	Stochastic — samples $\pi(a\|s)$	Deterministic — argmax $Q(s,a)$
Update style	On-policy (rollout buffer)	Off-policy (replay buffer)
Sample efficiency	Moderate — data discarded after K epochs	High — data reused from large buffer
Stability	High — clip prevents large steps	Moderate — sensitive to LR and C
Wall-clock speed	Slower per update (rollout needed)	Faster per update (any stored transition)
Our choice	**CartPole (continuous state, dense r)**	**FrozenLake (discrete, sparse r)**

Can PPO solve FrozenLake? Yes — but needs 3-5× more samples. Can DQN solve CartPole? Yes — with a discrete action wrapper, but less natural than PPO.

Reward Landscape — A Potential Energy Analogy

Discounted return $G_t = \Sigma_k \gamma^k r_{t+k}$ plays the role of **potential energy** evaluated at state s. The agent acts to maximise G_t, equivalently to **minimise U(s) = −V*(s)**.

CartPole — Dense Reward

$$V^*(\theta) \approx V_0 \cdot \exp(-\theta^2/\sigma^2)$$

Shape: Gaussian well — a 1D harmonic oscillator potential. Consistent with the linearised inverted pendulum.

V(0°) ≈ 500 · V decreases with |θ| · Sharp drop at ±12° (termination boundary)

Gradient ∇V exists everywhere → gradient ascent is efficient. This is why PPO converges in ~50k steps.

FrozenLake — Sparse Reward

$$U(s) = 0 \quad \text{for all } s \neq s_{goal}; \quad U(s_{goal}) = -1$$

Shape: Perfectly flat with a single delta-function minimum. ∇U ≈ 0 almost everywhere.

Gradient-based methods fail — there is no slope to climb. DQN's replay buffer collects the rare r=+1 transitions and amplifies them.

Reward shaping (gauge invariance): adding F(s,s') = γΦ(s')−Φ(s) preserves the optimal policy — choose Φ = Manhattan distance to goal.

Gauge invariance in RL: Ng, Harada & Russell (1999) proved that potential-based shaping preserves the optimal policy — the same freedom as adding ∇Λ to a classical force field without changing the equations of motion.

Phase Portrait & Lyapunov Stability

PPO Phase Portrait in $(\theta, \dot{\theta})$ Space

For small oscillations, the optimal LQR policy gives a linear decision boundary in $(\theta, \dot{\theta})$ space:

$$k_3\theta + k_4\dot{\theta} = 0 \quad \rightarrow \quad \theta = -(k_4/k_3)\dot{\theta}$$

Typical ratio $k_4/k_3 \approx 0.3$–0.5

The PPO phase portrait shows the same linear boundary — the neural network rediscovers the LQR solution without ever seeing the dynamics model.

Why the boundary tilts: if $\theta = 0$ but $\dot{\theta} > 0$ (pole upright but falling), the controller must push to counteract the angular acceleration — the tilt direction encodes this causal structure.

PPO learns the LQR optimal controller for CartPole empirically — without model knowledge. The decision boundary linearity is the signature of the Riccati solution.

Lyapunov Stability Analysis

Candidate: $V = \theta^2 + c\,\dot{\theta}^2 \quad (c = 1/g \approx 0.05\ s^2)$

This is the linearised mechanical energy of the pendulum.

Stability condition: $dV/dt < 0$ along trajectories.

mean dV/dt	≈ −0.001 to −0.0001	Small and negative — barely moves V
fraction dV < 0	≈ 55—70%	Most steps locally stabilising
Stable region	$\lvert\theta\rvert < 7°$	Well inside termination boundary
Near termination	mean dV/dt > 0	Policy can't hold stability at edge

Stability ≠ Robustness. The stable basin is smaller than the termination region. Practical implication: sim-to-real gap — small perturbations (model mismatch) can drive the state outside the stable basin.

FrozenLake with DQN — Deterministic vs Stochastic

Deterministic (is_slippery=False)

Steps	50 000
CPU time	~1–2 min
Success rate	90–100 %
Convergence	Irregular — agent may fail for many episodes before finding the goal

The MLP Q-table effectively memorises the optimal 16-state policy — equivalent to the tabular Q-learning solution from Week 5.

Stochastic (is_slippery=True)

Steps	200 000
CPU time	~5–8 min
Success rate	60–80 %
Theoretical max	~80 % (hole topology constraint)

Intended action taken only 1/3 of the time. Optimal policy cannot guarantee success. Stochastic FrozenLake = biased random walk with absorbing boundaries — solvable via Dirichlet BVP on the 4×4 lattice.

```
# Deterministic
env = gym.make('FrozenLake-v1', is_slippery=False)
model = DQN('MlpPolicy', env, verbose=0)
model.learn(total_timesteps=50_000)
```

```
# Stochastic
env = gym.make('FrozenLake-v1', is_slippery=True)
model = DQN('MlpPolicy', env, verbose=0, learning_starts=1000)
model.learn(total_timesteps=200_000)
```

Connections Across the Course

Week 5 RL Foundations	Bellman equation → DQN Q-values are the same V*(s) derived analytically. ε-greedy exploration in NumPy = exploration_fraction parameter in SB3. Q-learning discount factor γ* ≡ Grim Trigger cooperation threshold δ.
Weeks 1–4 Complex Systems	CartPole reward landscape ≈ harmonic potential (linearised pendulum). FrozenLake stochastic = random walk on 2D lattice — same physics as Boids diffusion. Phase transitions in policy quality: abrupt jump at a critical training budget.
Week 7 Multi-Agent Systems	PettingZoo environments expose the same Gymnasium-style step() / reset() API. Single-agent PPO/DQN fluency from Week 6 transfers directly to MARL. IRL (inverse RL) reads reward signals backward — Gymnasium is the common interface.
Week 8 LangGraph	LangGraph tools wrap environments the same way Gymnasium wraps simulators. RL agents choosing tool sequences = action selection in a Gymnasium environment. Gymnasium is the RL equivalent of a LangGraph tool registry.

Week 6 — Key Takeaways

Gymnasium 5-tuple

Terminated ≠ truncated. State vector = physics degrees of freedom.

FrozenLake = lattice walk

Sparse reward → replay buffer is essential; stochastic version is a Dirichlet BVP.

PPO

Clipped surrogate limits update size — trust region in policy space. On-policy, data reused K epochs.

LQR ↔ PPO

Neural network policy rediscovers the linear Riccati solution. Phase portrait linearity is the signature.

CartPole = inverted pendulum

Dense reward + continuous state → gradient always exists → PPO converges smoothly.

DQN

Replay buffer breaks temporal correlations. Target network stabilises bootstrap targets.

Reward landscape ↔ potential energy

Dense: harmonic well. Sparse: delta-function minimum. Reward shaping ↔ gauge invariance.

Lyapunov stability

Well-trained PPO is empirically Lyapunov-stable near equilibrium — but the stable basin < termination region.